Guide OAuth 2.0 Pennylane

Synthèse : flux, stockage des tokens, exemples Symfony / AdonisJS et backlog d'implémentation

Ce document est basé sur la documentation Pennylane « Implement OAuth 2.0 » (mise à jour du 10 juin 2026). Les recommandations qui n'en viennent pas sont signalées comme telles.

1. Ce que dit la documentation Pennylane

Sujet	Fait documenté par Pennylane
Accès OAuth	Inscription via l'équipe Partnerships (formulaire de demande de partenariat). Pennylane remet un Client ID et un Client Secret.
Qui utilise OAuth	Partenaires d'intégration (apps tierces) et cabinets comptables (API Firm). Les entreprises clientes utilisent un token API.
Flux	Authorization Code standard.
Access token	Durée de 24 h (`expires_in: 86400`).
Refresh token	Valable 90 jours, **à usage unique** (Refresh Token Rotation) : chaque refresh invalide l'ancien refresh token **et** l'ancien access token.
state	Optionnel côté Pennylane, mais à utiliser et valider systématiquement.
PKCE	Non mentionné dans la doc : ne pas y compter.
Révocation	`POST /oauth/revoke`, réponse 200 avec corps vide.
Secret perdu	Non récupérable : il faut créer une nouvelle app OAuth, ce qui peut prendre du temps.

2. OAuth ou token API ?

- **Application tierce ou accès à l'API Firm d'un cabinet** : OAuth 2.0 (ce guide).
- **Accès aux données de sa propre entreprise uniquement** : token API généré dans l'interface Pennylane, sans OAuth.

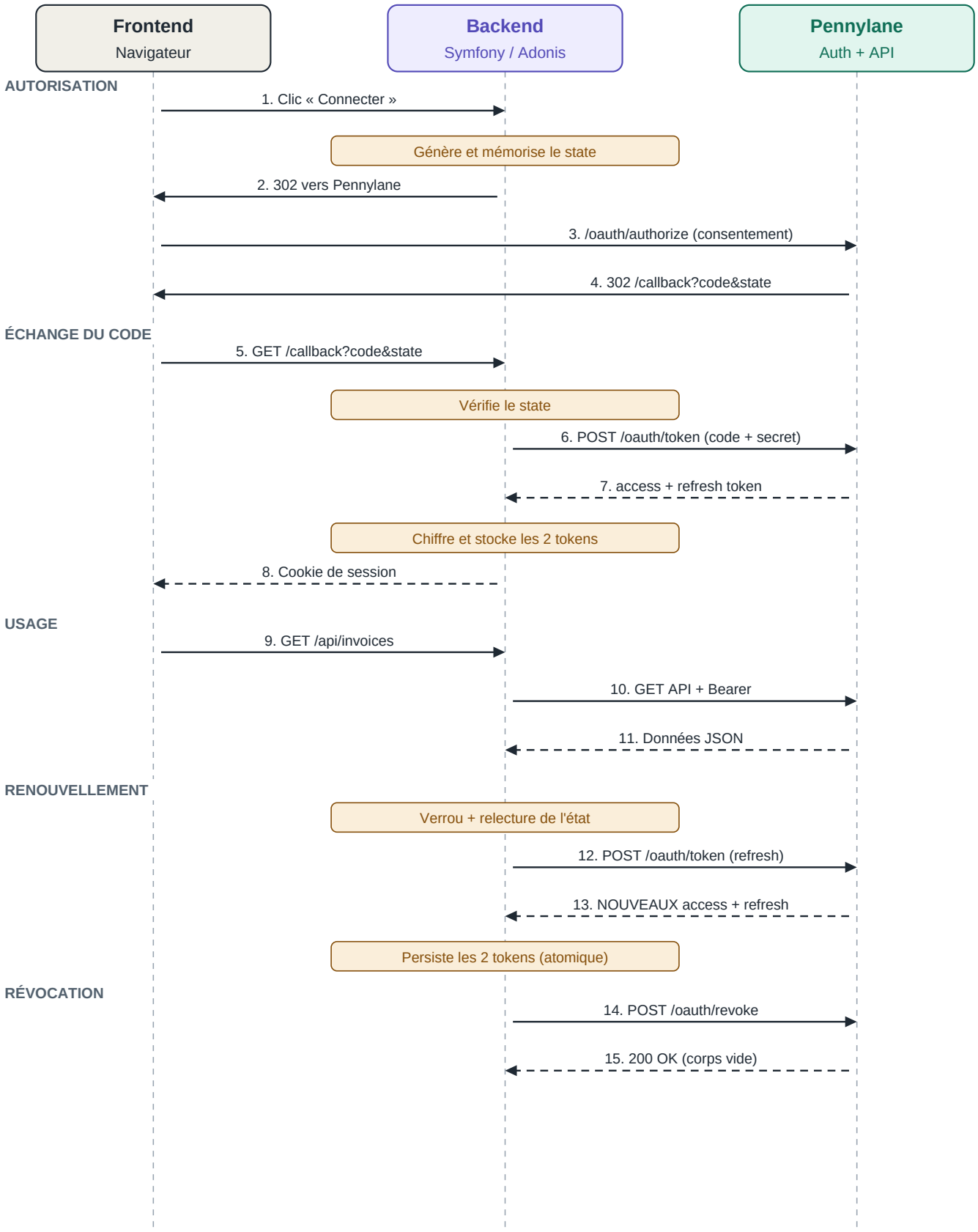
3. Les types de token

	Company token	Firm token
Portée	Une seule entreprise	Plusieurs entreprises clientes du cabinet
Exemple	`GET /api/external/v2/products` : produits de cette entreprise	`GET /api/external/v2/companies` : entreprises visibles ; les appels suivants se font dans le contexte d'une entreprise choisie
Visibilité	Limitée par les scopes	Limitée par les scopes **et** par les droits de l'utilisateur qui a accordé l'accès (entreprises confidentielles incluses ou non)

En résumé : les scopes définissent ce que l'on peut faire ; le type de token et l'utilisateur qui l'a accordé définissent sur quelles ressources.

4. Le flux complet

Le frontend ne fait que transporter des redirections. Tout ce qui est sensible (secret, code, tokens) se passe entre le backend et Pennylane. Les flèches en pointillés sont des réponses.



Lecture du diagramme

- **Autorisation (1 à 4)** : le frontend déclenche la connexion par un simple lien (vraie navigation, pas de `fetch`). Le backend génère le `state`, le met en session, puis redirige vers Pennylane. L'utilisateur choisit son entreprise ou son cabinet et accepte les scopes. Pennylane le renvoie vers le `redirect_uri` avec `code` et `state`.
- **Échange (5 à 8)** : le backend vérifie le `state`, puis échange le `code` contre les tokens (appel serveur à serveur avec le secret). Il les chiffre et les stocke, puis ouvre une session pour le navigateur.
- **Usage (9 à 11)** : le frontend appelle uniquement le backend. Celui-ci retrouve le token et appelle l'API Pennylane avec `Authorization: Bearer`.
- **Renouvellement (12 et 13)** : après 24 h, le backend utilise le refresh token. Pennylane renvoie **deux nouveaux tokens** ; les anciens sont morts. Il faut donc un verrou, une relecture de l'état et une écriture atomique des deux tokens.
- **Révocation (14 et 15)** : quand l'accès n'est plus utile, le backend appelle `/oauth/revoke`, puis supprime les tokens locaux.

Endpoints Pennylane

Étape	Appelant -> cible	Endpoint
Autorisation	Navigateur -> Pennylane	GET https://app.pennylane.com/oauth/authorize
Échange du code	Backend -> Pennylane	POST https://app.pennylane.com/oauth/token
Renouvellement	Backend -> Pennylane	POST https://app.pennylane.com/oauth/token
Révocation	Backend -> Pennylane	POST https://app.pennylane.com/oauth/revoke

Appel	Paramètres
/oauth/authorize (GET)	client_id (oui), redirect_uri (oui), response_type=code (oui), scope séparés par des espaces (oui), state (non, mais à faire)
/oauth/token (code)	Corps : client_id, client_secret, code, redirect_uri (identique à l'autorisation), grant_type=authorization_code
/oauth/token (refresh)	Corps : client_id, client_secret, refresh_token, grant_type=refresh_token
/oauth/revoke	Corps : client_id, client_secret, token (access ou refresh)

Réponse de `/oauth/token` : `{ access_token, token_type: "Bearer", expires_in: 86400, refresh_token }`

5. Où vit chaque élément

Élément	Où	Durée	Remarque
Client ID	Config backend	Permanent	Non secret
Client Secret	Gestionnaire de secrets / variable d'environnement	Permanent	Jamais en base, ni dans Git, ni côté navigateur
state	Session serveur	Quelques minutes	Comparé puis supprimé au retour
code	Nulle part	Très court	Échangé immédiatement
Access token	Base de données, chiffré	24 h	Invalidé dès qu'un refresh est utilisé
Refresh token	Base de données, chiffré	90 jours	Usage unique, remplacé à chaque refresh
Session utilisateur	Cookie HttpOnly, Secure, SameSite=Lax	Celle de l'app	Seule chose que voit le frontend

Règle d'or : le frontend ne voit jamais un token Pennylane. Même pour une SPA, elle appelle le backend avec `credentials: 'include'`. Ne jamais stocker un token dans `localStorage`.

Modèle de données proposé

```
pennylane_connection
  id
  owner_id              -> utilisateur / client propriétaire
  token_type            -> 'company' | 'firm'
  access_token          (chiffré)
  refresh_token         (chiffré)
  access_expires_at
  refresh_expires_at    (création + 90 jours, voir point à confirmer n°3)
  scopes
  status                -> 'active' | 'reconnect_required' | 'revoked'
  created_at / updated_at
```

6. Exemples de code

Les noms de scopes (`scope_a`, `scope_b`) sont des placeholders : voir la liste fournie par Pennylane.

6.1 Symfony (league/oauth2-client)

```
$provider = new GenericProvider([
    'clientId'                => $_ENV['PENNYLANE_CLIENT_ID'],
    'clientSecret'            => $_ENV['PENNYLANE_CLIENT_SECRET'],
    'redirectUri'             => $_ENV['PENNYLANE_REDIRECT_URI'],
    'urlAuthorize'            => 'https://app.pennylane.com/oauth/authorize',
    'urlAccessToken'          => 'https://app.pennylane.com/oauth/token',
    'urlResourceOwnerDetails' => 'https://app.pennylane.com/oauth/token', // requis, jamais appel\u00e9
    'scopeSeparator'          => ' ', // Pennylane attend des espaces (la lib met une virgule)
]);
```

Piège : sans `scopeSeparator`, les scopes sont séparés par des virgules et Pennylane les rejette.

```
#[Route('/connect/pennylane')]
public function connect(Request $request): Response
{
    $url = $this->provider->getAuthorizationUrl(['scope' => ['scope_a', 'scope_b']]);
    $request->getSession()->set('oauth_state', $this->provider->getState());
    return $this->redirect($url);
}

#[Route('/connect/pennylane/callback')]
public function callback(Request $request): Response
{
    if ($request->query->has('error')) {            // l'utilisateur a refus\u00e9
        return $this->redirectToRoute('settings', ['pennylane' => 'denied']);
    }
    $expected = $request->getSession()->remove('oauth_state');
    if (!$expected || $expected !== $request->query->get('state')) {
        throw $this->createAccessDeniedException('State invalide');
    }
    $token = $this->provider->getAccessToken('authorization_code', [
        'code' => $request->query->get('code'),
    ]);
    $this->connections->createOrReplace($this->getUser(), $token);
    return $this->redirectToRoute('dashboard');
}
```

Service de token (rotation du refresh token)

```
public function getValidAccessToken(PennylaneConnection $conn): string
{
    if (!$conn->expiresWithin(300)) {               // marge de 5 min
        return $conn->getAccessToken();
    }
    $lock = $this->lockFactory->createLock('pennylane_refresh_' . $conn->getId(), 30);
    $lock->acquire(true);                           // attend si un autre process refresh
    try {
        $this->em->refresh($conn);                  // peut-\u00eatre d\u00e9j\u00e0 renouvel\u00e9
        if ($conn->expiresWithin(300)) {
            try {
                $new = $this->provider->getAccessToken('refresh_token', [
                    'refresh_token' => $conn->getRefreshToken(),
                ]);
            } catch (IdentityProviderException $e) {
                $conn->setStatus('reconnect_required');
                $this->em->flush();
                throw new PennylaneReconnectRequired();
            }
            $conn->updateFrom($new);                // access + refresh + dates
            $this->em->flush();                     // AVANT de rel\u00e2cher le verrou
```

```
        }
    } finally {
        $lock->release();
    }
    return $conn->getAccessToken();
}
```

Le store de `symfony/lock` doit être partagé entre tous les processus (Redis, PostgreSQL, Doctrine DBAL). Le store `flock` par défaut ne protège que sur une seule machine.

Révocation

```
$this->httpClient->request('POST', 'https://app.pennylane.com/oauth/revoke', [
    'body' => [                                     // tableau = form-urlencoded
        'client_id'     => $_ENV['PENNYLANE_CLIENT_ID'],
        'client_secret' => $_ENV['PENNYLANE_CLIENT_SECRET'],
        'token'         => $conn->getRefreshToken(),
    ],
]);                                                 // 200 + corps vide si OK
```

6.2 AdonisJS (simple-oauth2)

```
const client = new AuthorizationCode({
  client: { id: env.get('PENNYLANE_CLIENT_ID'), secret: env.get('PENNYLANE_CLIENT_SECRET') },
  auth: {
    tokenHost: 'https://app.pennylane.com',
    authorizePath: '/oauth/authorize',
    tokenPath: '/oauth/token',
  },
  options: { authorizationMethod: 'body' }, // id/secret dans le corps, pas en Basic Auth
})
```

Piège : par défaut, `simple-oauth2` envoie les identifiants dans un en-tête `Authorization: Basic`, alors que la doc Pennylane les place dans le corps.

```
async connect({ session, response }: HttpContext) {
  const state = randomBytes(16).toString('hex')
  session.put('oauth_state', state)
  return response.redirect(client.authorizeURL({
    redirect_uri: env.get('PENNYLANE_REDIRECT_URI'),
    scope: 'scope_a scope_b',                // s\u00e9par\u00e9s par des espaces
    state,
  }))
}

async callback({ request, session, auth, response }: HttpContext) {
  if (request.input('error')) return response.redirect('/settings?pennylane=denied')
  if (session.pull('oauth_state') !== request.input('state')) return response.forbidden()

  const token = await client.getToken({
    code: request.input('code'),
    redirect_uri: env.get('PENNYLANE_REDIRECT_URI'),
  })
  await PennylaneConnection.createOrReplaceFor(auth.user!, token.token)
  return response.redirect('/dashboard')
}

async getValidAccessToken(connectionId: number): Promise<string> {
  return db.transaction(async (trx) => {
    const conn = await PennylaneConnection.query({ client: trx })
      .where('id', connectionId).forUpdate().firstOrFail()      // s\u00e9rialise les refresh

    if (conn.accessExpiresAt <= DateTime.now().plus({ minutes: 5 })) {
      try {
        const refreshed = await client
          .createToken({ refresh_token: encryption.decrypt(conn.refreshToken) })
          .refresh()
        conn.applyTokens(refreshed.token)        // access + refresh + dates, chiffr\u00e9s
        await conn.useTransaction(trx).save()
      } catch (e) {
        conn.status = 'reconnect_required'
        await conn.useTransaction(trx).save()
        throw new PennylaneReconnectRequired()
      }
    }
    return encryption.decrypt(conn.accessToken)
  })
}

await fetch('https://app.pennylane.com/oauth/revoke', {
```

```
    method: 'POST',
    body: new URLSearchParams({
      client_id: env.get('PENNYLANE_CLIENT_ID'),
      client_secret: env.get('PENNYLANE_CLIENT_SECRET'),
      token: encryption.decrypt(conn.refreshToken),
    }),
  })
```

6.3 Côté frontend

```
<a href="/connect/pennylane">Connecter Pennylane</a>

const res = await fetch('/api/invoices', { credentials: 'include' });
if (res.status === 401) { /* proposer de reconnecter Pennylane */ }
```

7. La rotation du refresh token

Chaque utilisation d'un refresh token l'invalide immédiatement, ainsi que l'ancien access token. Règles de conception :

- **Un seul refresh à la fois par connexion** : verrou (Symfony) ou `FOR UPDATE` (Adonis). Deux refresh avec le même token, et le second échoue.
- **Relire l'état après avoir obtenu le verrou** : un autre processus a pu renouveler le token entre-temps.
- **Persister le nouveau refresh token avant tout le reste** : un crash entre la réponse de Pennylane et l'écriture en base brûle l'ancien token et perd le nouveau.
- **Sur un 401** : relire le token en base et rejouer la requête une seule fois, puis passer à `reconnect_required`.
- **Pas de refresh parallèle** entre un job et une requête web sans verrou commun.
- **Refresh token expiré (90 jours)** : le seul recours est de refaire passer l'utilisateur par le flux d'autorisation.
- **Recommandation (hors doc)** : une tâche planifiée qui renouvelle les connexions peu utilisées bien avant 90 jours, sinon elles meurent silencieusement.

8. Gestion des erreurs

Situation	Détection	Réaction
L'utilisateur refuse	?error=... sur le callback, pas de code	Message clair, aucun échange de token
state incohérent	Comparaison avec la session	403, pas d'échange
Refresh refusé	Erreur sur /oauth/token	Statut reconnect_required, bouton « Reconnecter »
401 sur l'API	Réponse de l'API	Relire le token, rejouer une fois, sinon reconnect_required
Erreur réseau pendant un refresh	Timeout, 5xx	Ne pas conclure à une perte d'accès : réessayer plus tard (le token a pu être consommé, à tester)

9. Points à confirmer avec Pennylane

La doc ne répond pas à ces questions. À poser à l'équipe Partnerships lors de l'enregistrement :

1. Noms exacts des scopes disponibles et ceux nécessaires.
2. `redirect_uri` autorisées : déclaration préalable, plusieurs possibles (local, staging, production) ?
3. Les 90 jours du refresh token sont-ils renouvelés à chaque rotation ou comptés depuis la première autorisation ?
4. Firm token : comment sélectionner l'entreprise cible sur les appels suivants (en-tête, paramètre, chemin) ?
5. Comment identifier l'entreprise liée à un company token après l'échange (la réponse n'a pas d'identifiant d'entreprise) ?
6. Existence d'un environnement de test ou sandbox, identifiants distincts par environnement ?
7. Révoquer l'access token révoque-t-il aussi le refresh token, et inversement ?
8. PKCE : supporté ou non ?
9. Limites de débit de l'API.

10. Backlog d'implémentation

Priorité : P0 bloquant, P1 important, P2 amélioration. Taille : S (moins d'une demi-journée), M (environ une journée), L (plusieurs jours). Ordre recommandé : epics 0 à 3, puis 4 **avant toute mise en usage réel** (la fiabilité de tout le reste en dépend). Les epics 5, 6 et 7 peuvent avancer en parallèle une fois l'epic 3 terminé.

Epic 0 — Prérequis (hors code)

#	Tâche	Critère d'acceptation	P	Taille
0.1	Contacter Partnerships via le formulaire, préciser le cas d'usage (partenaire ou cabinet)	Demande envoyée, interlocuteur identifié	P0	S
0.2	Poser les questions de la section 9	Réponses écrites et archivées	P0	S
0.3	Recevoir Client ID et Secret, les stocker immédiatement dans un gestionnaire de secrets	Secret dans le coffre, jamais dans Git ni en clair dans un ticket	P0	S
0.4	Lister les scopes strictement nécessaires	Liste validée (moindre privilège)	P0	S

Epic 1 — Fondations

#	Tâche	Critère d'acceptation	P	Taille
1.1	Variables d'environnement par environnement (ID, secret, redirect URI, scopes)	L'app démarre avec la config injectée, aucun secret dans le dépôt	P0	S
1.2	Installer la bibliothèque OAuth avec séparateur de scopes / méthode d'authentification corrigés	Test unitaire : l'URL d'autorisation contient les scopes séparés par des espaces	P0	S
1.3	Entité / modèle pennylane_connection et migration	Migration appliquée, champs de la section 5	P0	M
1.4	Chiffrer les tokens au repos	Lecture directe en base illisible, déchiffrement OK dans l'app	P0	M

Epic 2 — Autorisation

#	Tâche	Critère d'acceptation	P	Taille
2.1	Route /connect/pennylane : génération et stockage du state, redirection	Redirection vers /oauth/authorize avec tous les paramètres	P0	S
2.2	Route de callback : gestion de error, vérification du state, échange du code	Tokens obtenus et persistés, state à usage unique	P0	M
2.3	Rattacher la connexion au bon propriétaire et au bon type (company ou firm)	Une connexion par entreprise ou cabinet, doublons gérés	P0	M
2.4	Bouton « Connecter Pennylane » et retour d'état (succès, refus)	Parcours complet testé manuellement	P0	S

Epic 3 — Appels API

#	Tâche	Critère d'acceptation	P	Taille
3.1	Service getValidAccessToken() avec marge de 5 minutes	Un token proche de l'expiration est renouvelé automatiquement	P0	M
3.2	Client HTTP Pennylane qui injecte le Bearer token	Aucun autre code ne lit directement les tokens	P0	M
3.3	Gestion du 401 : relecture et un seul rejeu	Test avec 401 simulé	P1	M
3.4	Contexte entreprise pour les firm tokens	Selon la réponse à la question 4 de la section 9	P1	M

Epic 4 — Rotation (le plus critique)

#	Tâche	Critère d'acceptation	P	Taille
4.1	Verrou par connexion (store partagé en Symfony, FOR UPDATE en Adonis)	Deux refresh simultanés = un seul appel à /oauth/token	P0	M
4.2	Relecture de l'état après acquisition du verrou	Le second appelant utilise le token déjà renouvelé	P0	S
4.3	Persister access et refresh token en une seule écriture, avant libération du verrou	Test avec crash simulé juste après le refresh	P0	M
4.4	Test de concurrence (10 requêtes parallèles sur un token expiré)	Une seule rotation, aucune requête en erreur	P0	M
4.5	Tâche planifiée de maintien des connexions inactives	Aucune connexion ne dépasse 60 jours sans refresh	P1	M

Epic 5 — Déconnexion et erreurs

#	Tâche	Critère d'acceptation	P	Taille
5.1	Passage en reconnect_required lors d'un refresh refusé	Statut visible, aucun retry en boucle	P0	S
5.2	Bandeau / bouton « Reconnecter » côté frontend	L'utilisateur relance le flux en un clic	P0	S
5.3	Fonction « Déconnecter » : appel à /oauth/revoke puis suppression locale	200 reçu, tokens effacés, statut revoked	P1	M
5.4	Distinguer erreur réseau et erreur d'autorisation	Une erreur réseau ne déclenche pas reconnect_required	P1	M
5.5	Notification quand une connexion passe en reconnect_required	Message reçu par le propriétaire	P2	M

Epic 6 — Sécurité et observabilité

#	Tâche	Critère d'acceptation	P	Taille
6.1	Masquer tokens et secret dans les logs et outils d'erreurs	Recherche dans les logs de test : aucune occurrence	P0	S
6.2	Cookie de session HttpOnly, Secure, SameSite=Lax	Vérifié dans les en-têtes de réponse	P0	S
6.3	Métriques et alertes : taux d'échec de refresh, connexions en erreur	Tableau de bord et alerte configurés	P1	M
6.4	Runbook en cas de fuite du secret (perte = recréer une app OAuth)	Runbook rédigé	P1	S

Epic 7 — Tests et mise en production

#	Tâche	Critère d'acceptation	P	Taille
7.1	Tests unitaires du service de token avec Pennylane simulé	Couverture des cas de la section 8	P0	M
7.2	Test de bout en bout sur l'environnement de test, s'il existe	Connexion, appel API, refresh, révocation validés	P1	L
7.3	Déclarer les redirect_uri de production auprès de Pennylane	URI validée avant le déploiement	P0	S
7.4	Déploiement avec secrets de production, vérification du premier parcours réel	Une connexion réelle fonctionne	P0	M